

Programação Orientada a Objetos



Aula 6 - Coleções (Collections)

Prof. Guilherme Guerino



Introdução

- Uma coleção é uma estrutura de dados que permite armazenar vários objetos.
 - Em Java , a coleção também é um objeto.
- As operações que podem ser feitas em coleções variam mas normalmente incluem:
 - Adição de elementos;
 - Remoção de elementos;
 - Acesso aos elementos;
 - Pesquisa de elementos;



Introdução

- Dependendo da forma de fazer as 4 operações básicas (adição, remoção, acesso e pesquisa), teremos vários tipos de coleções
- Os três grandes tipos de coleções são:
 - List;
 - Set;
 - Map.

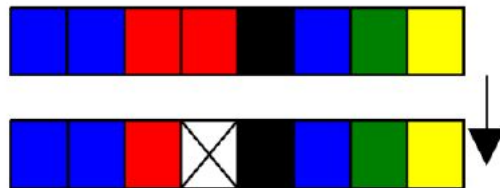


Introdução

- Manipular arrays é bastante trabalhoso. Essa dificuldade aparece em diversos momentos:
 - Não podemos redimensionar um array em Java;
 - É impossível buscar diretamente por um determinado elemento cujo índice não se sabe;
 - Não conseguimos saber quantas posições do array já foram “populadas”



Introdução



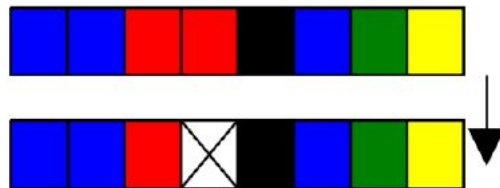
Retire a quarta Conta

`conta[3] = null;`

- Supondo que os dados armazenados representem contas, o que acontece quando precisarmos inserir uma nova conta no banco?
 - Precisaremos procurar por um espaço vazio?
 - Guardaremos em alguma estrutura de dados externa, as posições vazias?
 - E se não houver espaço vazio?
 - Teríamos de criar um array maior e copiar os dados do antigo para ele?



Introdução



Retire a quarta Conta

`conta[3] = null;`

- Supondo que os dados armazenados representem contas, o que acontece quando precisarmos inserir uma nova conta no banco?
 - Como posso saber quantas posições estão sendo usadas no array?
 - Vou precisar sempre percorrer o array inteiro para conseguir essa informação?

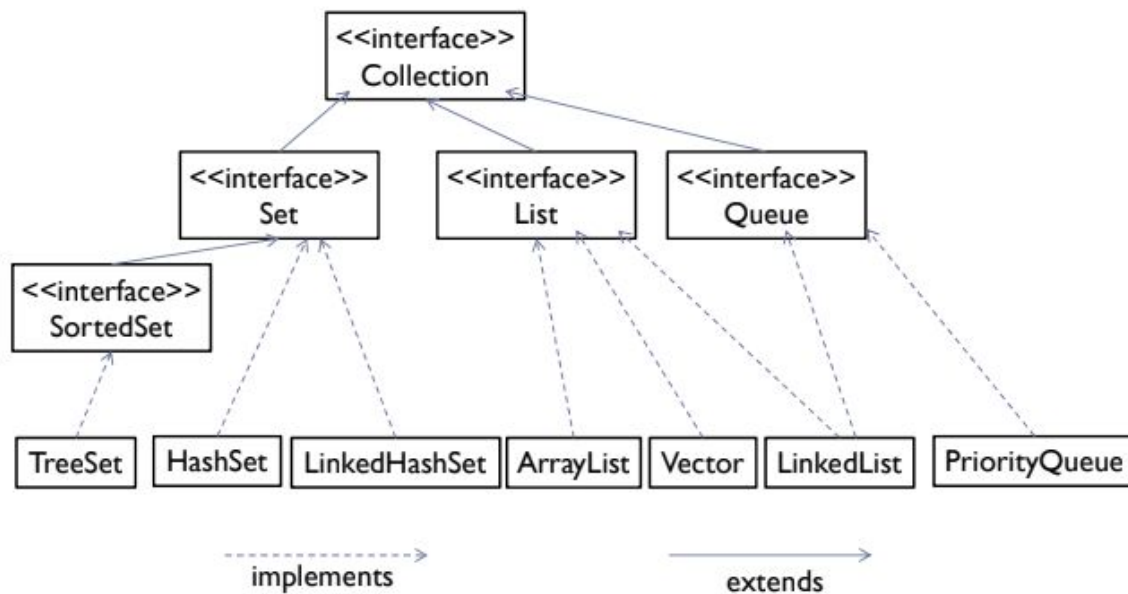


Coleções

- Com esses e outros objetivos em mente, foi criado um conjunto de classes e interfaces conhecido como Collections Framework.
- A API do Collections é robusta e possui diversas classes que representam estruturas de dados avançadas.
 - Por exemplo, não é necessário reinventar a roda e criar uma lista ligada, mas sim utilizar aquela que já tem disponibilizado no *framework*.

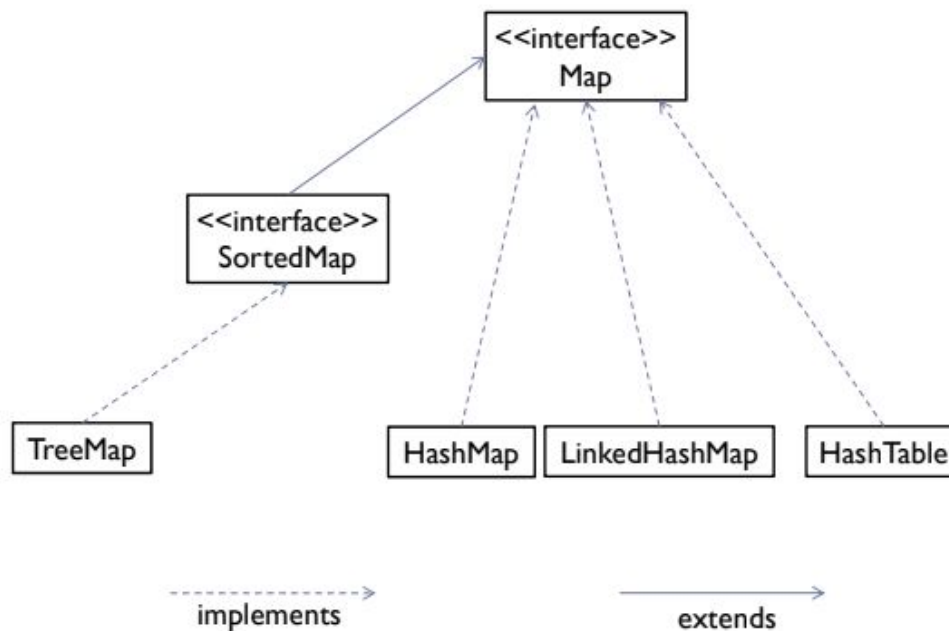


Coleções





Coleções





List

- Coleção de elementos organizados em uma ordem linear, onde cada elemento tem um antecessor e um sucessor.
 - Usado quando a sequência importa.
- Coleções que sabem a respeito da posição de seus índices.
 - Um List sabe quando algo está na lista.
- Classes que implementam a interface List: **ArrayList**, Vector, LinkedList.



Exemplo ArrayList

```
1 package arrayscolecoes;
2
3 import java.util.ArrayList;
4 import java.util.Arrays;
5
6 public class Main {
7
8     public static void main(String[] args) {
9         ArrayList<String> alunos = new ArrayList();
10        alunos.add("Renan");
11        System.out.println(Arrays.toString(alunos.toArray()));
12        alunos.add("Joao");
13        System.out.println(Arrays.toString(alunos.toArray()));
14        alunos.add("Edson");
15        System.out.println(Arrays.toString(alunos.toArray()));
16        System.out.println("Aluno no index 2: "+alunos.get(2));
17        System.out.println("Index do aluno Renan: "+alunos.indexOf("Renan"));
18    }
19 }
```

! Saida - ArraysColecoes (run)



run:

[Renan]

[Renan, Joao]

[Renan, Joao, Edson]

Aluno no index 2: Edson

Index do aluno Renan: 0

CONSTRUÍDO COM SUCESSO (tempo total: 1 segundo)



Exemplo ArrayList

```
2
3 import java.util.ArrayList;
4 import java.util.Arrays;
5
6 public class Main {
7
8     public static void main(String[] args) {
9         ArrayList<Aluno> alunos = new ArrayList();
10        Aluno a = new Aluno("Renan");
11        Aluno b = new Aluno("Joao");
12        Aluno c = new Aluno("Edson");
13        alunos.add(a);
14        alunos.add(b);
15        alunos.add(c);
16        alunos.add(a);
17        for (int i = 0; i < alunos.size(); i++) {
18            System.out.println(alunos.get(i).getNome());
19        }
20        System.out.println("Aluno no index 2: "+alunos.get(2).getNome());
21        System.out.println("Index do aluno Joao: "+alunos.indexOf(b));
22    }
23 }
```

Saida - ArraysColecoes (run)



run:

Renan

Joao

Edson

Renan

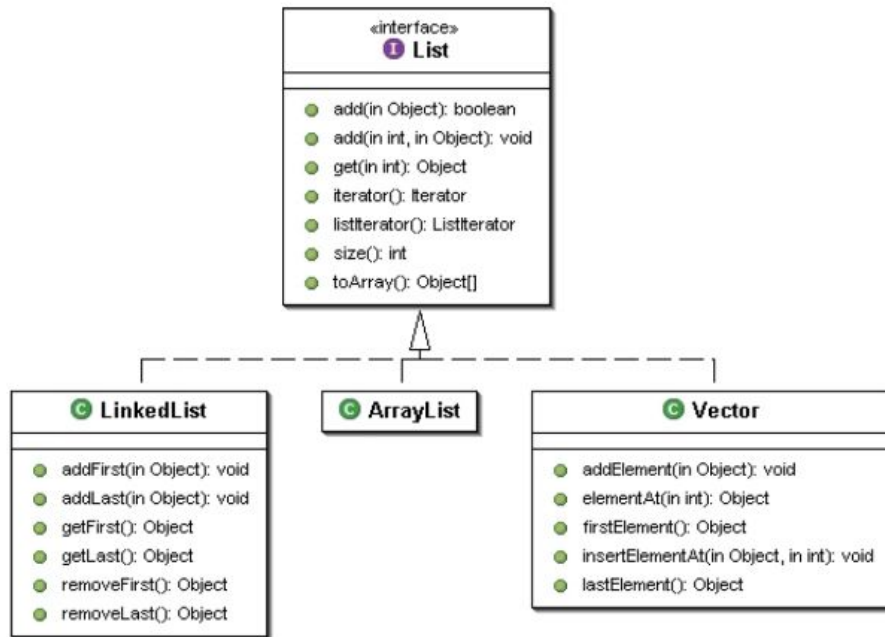
Aluno no index 2: Edson

Index do aluno Joao: 1

CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)



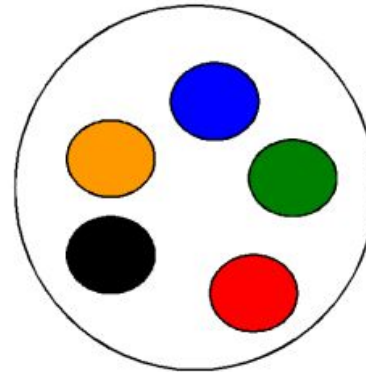
List





Set (conjunto)

- Um conjunto (Set) funciona de forma análoga aos conjuntos da matemática, ele é uma coleção que não permite elementos duplicados.



Possíveis ações em um conjunto:

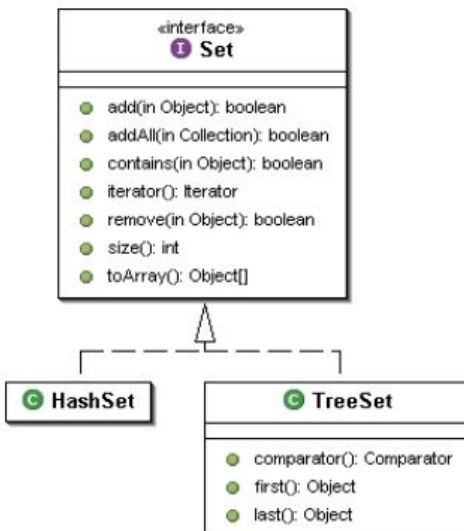
- A camiseta Azul está no conjunto?
- Remova a camiseta Azul.
- Adicione a camiseta Vermelha.
- Limpe o conjunto.

- **Não existem elementos duplicados!**
- **Ao percorrer um conjunto, sua ordem não é conhecida!**



Set (conjunto)

- Um conjunto é representado pela interface Set e tem como suas principais implementações as classes HashSet e TreeSet.





Set (conjunto)

```
Set<String> cargos = new HashSet<>();

cargos.add("Gerente");
cargos.add("Diretor");
cargos.add("Presidente");
cargos.add("Secretária");
cargos.add("Funcionário");
cargos.add("Diretor"); // repetido!

// imprime na tela todos os elementos
System.out.println(cargos);
```

- Aqui, o segundo Diretor não será adicionado e o método add lhe retornará false.



Set (conjunto)

- O uso de um Set pode parecer ***desvantajoso***:
 - Ele não armazena a ordem, e não aceita elementos repetidos.
 - Não há métodos que trabalham com índices, como o `get(int)` que as listas possuem.
- ***A grande vantagem do Set é que***
 - Existem implementações, como a `HashSet`, que possui uma performance incomparável com as `Lists` quando usado para pesquisa (método `contains` por exemplo).



Collection

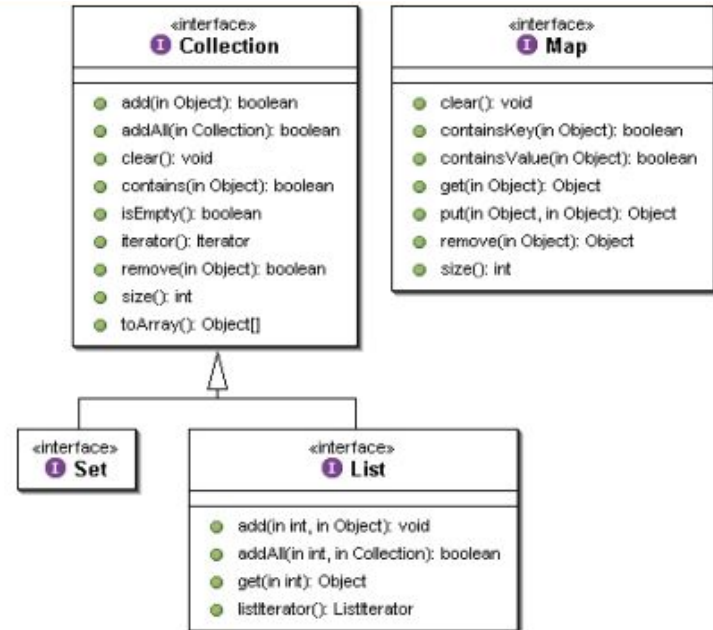
- As coleções têm como base a interface Collection, que define métodos para adicionar e remover um elemento, e verificar se ele está na coleção, entre outras operações, como mostra a tabela a seguir:

<code>boolean add(Object)</code>	Adiciona um elemento na coleção. Como algumas coleções não suportam elementos duplicados, este método retorna true ou false indicando se a adição foi efetuada com sucesso.
<code>boolean remove(Object)</code>	Remove determinado elemento da coleção. Se ele não existia, retorna false.
<code>int size()</code>	Retorna a quantidade de elementos existentes na coleção.
<code>boolean contains(Object)</code>	Procura por determinado elemento na coleção, e retorna verdadeiro caso ele exista. Esta comparação é feita baseando-se no método equals() do objeto, e não através do operador ==.
<code>Iterator iterator()</code>	Retorna um objeto que possibilita percorrer os elementos daquela coleção.



Collection

- Uma coleção pode implementar diretamente a interface Collection, porém normalmente se usa uma das duas sub interfaces mais famosas: justamente Set e List.





Percorrendo Collection

- Como percorrer os elementos de uma coleção?
 - Se for uma lista, podemos sempre utilizar um laço for, invocando o método get para cada elemento.
 - Mas e se a coleção não permitir indexação?
- Por exemplo, um Set não possui um método para pegar o primeiro, o segundo ou o quinto elemento do conjunto, já que um conjunto não possui o conceito de “ordem”



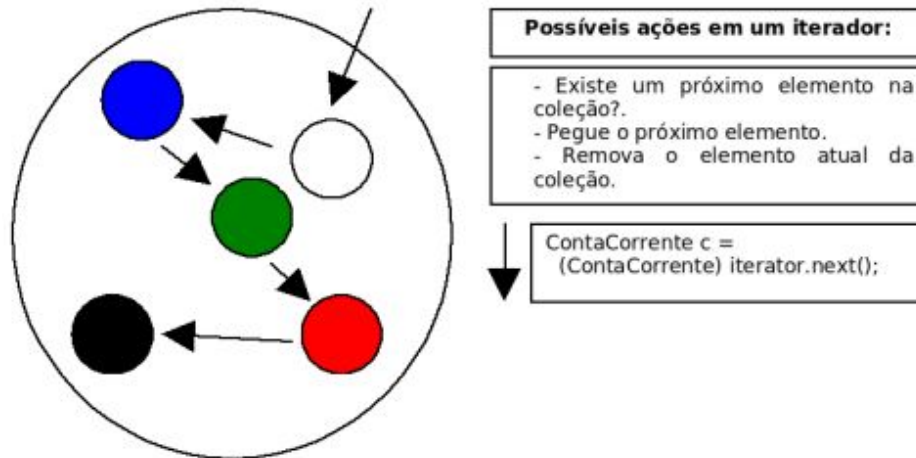
Percorrendo Collection

- Como percorrer os elementos de uma coleção?
 - Se for uma lista, podemos sempre utilizar um laço for, invocando o método get para cada elemento.
 - Mas e se a coleção não permitir indexação?
- Por exemplo, um Set não possui um método para pegar o primeiro, o segundo ou o quinto elemento do conjunto, já que um conjunto não possui o conceito de “ordem”
- Podemos usar o enhanced-for (o “foreach”) para percorrer qualquer Collection sem nos preocupar com isso.



Percorrendo Collection

- Toda coleção fornece acesso a um iterator, um objeto que implementa a interface Iterator, que conhece internamente a coleção e dá acesso a todos os seus elementos.





Percorrendo Collection

- Primeiro criamos um Iterator que entra na coleção. A cada chamada do método next, o Iterator retorna o próximo objeto do conjunto. Um iterator pode ser obtido com o método iterator() de Collection, por exemplo:

```
Set<String> conjunto = new HashSet<>();  
conjunto.add("item 1");  
conjunto.add("item 2");  
conjunto.add("item 3");  
  
Iterator<String> i = conjunto.iterator();  
while (i.hasNext()) {  
    // recebe a palavra  
    String palavra = i.next();  
    System.out.println(palavra);  
}
```



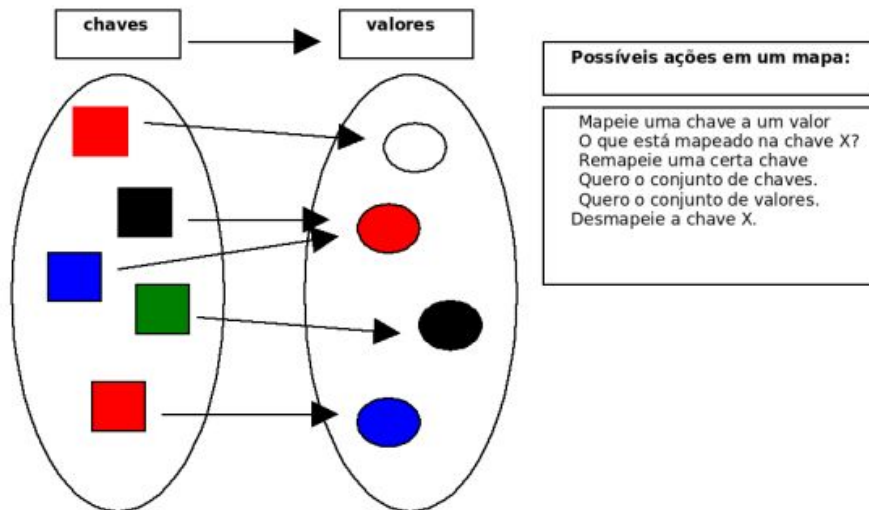
Map

- Muitas vezes queremos buscar rapidamente um objeto dado alguma informação sobre ele.
- Um exemplo seria:
 - Dada a placa do carro, obter todos os dados do carro.
 - Poderíamos utilizar uma lista para isso e percorrer todos os seus elementos, mas isso pode ser péssimo para a performance, mesmo para listas não muito grandes.



Map

- Um mapa é composto por um conjunto de associações entre um objeto chave a um objeto valor.





Map

- O método `put( object, object )` da interface `Map` recebe a chave e o valor de uma nova associação.
- Para saber o que está associado a um determinado objeto-chave, passa-se esse objeto no método `get(Object)`.
- Essas são as duas operações principais e mais frequentes realizadas sobre um mapa



Map

```
2
3 import java.util.HashMap;
4 import java.util.Iterator;
5 import java.util.Map;
6
7 public class Main {
8
9     public static void main(String[] args) {
10         Map<String, String> alunos = new HashMap();
11         alunos.put("K1", "Renan");
12         alunos.put("K2", "Joao");
13         alunos.put("K3", "Edson");
14         alunos.put("K4", "Joao");
15
16         for (String key : alunos.keySet()) {
17             System.out.println(key + " = " + alunos.get(key));
18         }
19         System.out.println(alunos.containsValue("Renan"));
20         System.out.println(alunos.containsKey("K5"));
21         System.out.println(alunos.size());
22     }
23 }
```

:: Saída - ArraysColecoes (run)

run:
K1 = Renan
K2 = Joao
K3 = Edson
K4 = Joao
true
false
4
CONSTRUÍDO COM SUCESSO



Map

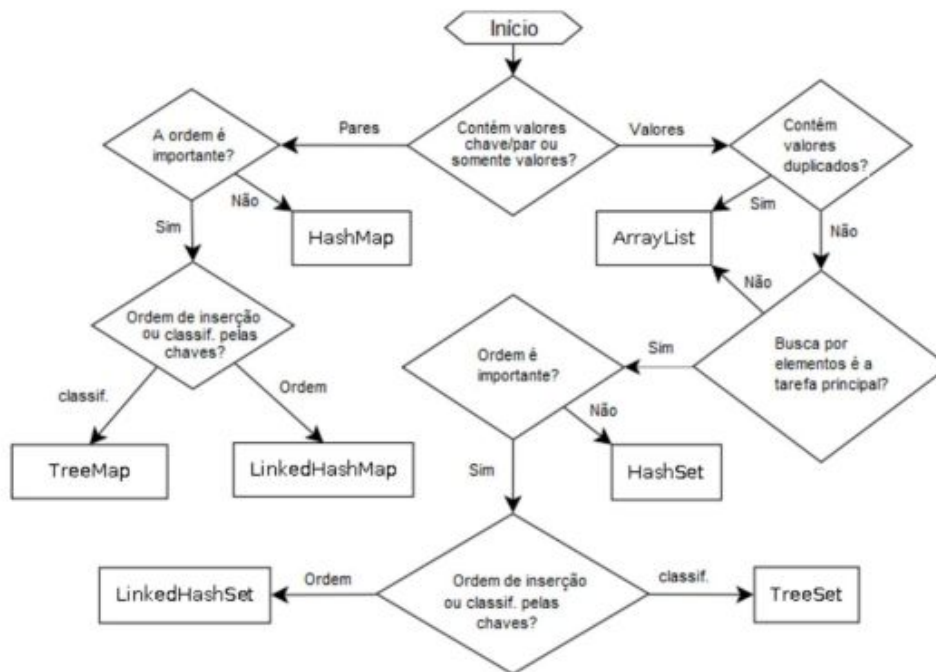
```
3 import java.util.HashMap;
4 import java.util.Iterator;
5 import java.util.Map;
6
7 public class Main {
8
9     public static void main(String[] args) {
10         Map<String, String> alunos = new HashMap();
11         alunos.put("K1", "Renan");
12         alunos.put("K2", "Joao");
13         alunos.put("K3", "Edson");
14         alunos.put("K4", "Joao");
15         alunos.put("K3", null);
16
17         for (String key : alunos.keySet()) {
18             System.out.println(key + " = " + alunos.get(key));
19         }
20         System.out.println(alunos.containsValue("Renan"));
21         System.out.println(alunos.containsKey("K5"));
22         System.out.println(alunos.size());
23     }
24 }
```

Saida - ArraysColecoes (run)

```
run:
K1 = Renan
K2 = Joao
K3 = null
K4 = Joao
true
false
4
CONSTRUÍDO COM SUCESSO
```



Qual coleção usar?





Documentações

- List
 - <https://docs.oracle.com/javase/8/docs/api/java/util/List.html>
- Set
 - <https://docs.oracle.com/javase/8/docs/api/java/util/Set.html>
- Map
 - <https://docs.oracle.com/javase/8/docs/api/java/util/Map.html>

Dúvidas?

